WikiSkill

에이전트 경험을 지속 지식으로 컴파일하는 Skill Evolution

> **15분 발표용 한국어 스토리텔링 대본**
>
> 논문 원문과 앞서 작성한 발표 초안을 바탕으로, 발표자가 그대로 읽거나 슬라이드에 맞춰 축약할 수 있도록 구성한 실전용 문서입니다.

발표를 관통하는 한 문장

"WikiSkill은 에이전트에게 더 좋은 Skill 하나를 주는 방법이 아니라, 경험을 잊지 않고 다음 Skill의 재료로 축적하는 방법입니다."

기본 정보

항목	내용
논문	WikiSkill: Compiling Agent Experience into Persistent Knowledge for Skill Evolution
발표 시간	약 15분
핵심 프레임	Raw Trace → Wiki → Skill
핵심 철학	Skill은 rollback할 수 있지만, Wiki는 reset하지 않는다.

> **사용 방법**
>
> 문장 앞의 [발표자 큐]는 슬라이드 지시입니다. 굵게 강조된 핵심 문장은 속도를 늦추고, 수식·표 페이지에서는 숫자를 전부 읽기보다 화살표와 추세를 먼저 설명하면 15분 안에 자연스럽게 들어옵니다.

전체 흐름 | 15분 발표 타임라인

이 발표는 '왜 필요한가 → 무엇을 바꾸는가 → 효과가 있는가 → 왜 효과가 있는가'의 순서로 전개합니다.

시간	구간	청중에게 남길 메시지
0:00–1:10	오프닝	AI 에이전트에게도 경력이 쌓이는가?
1:10–2:30	문제 정의	기존 Skill Evolution은 배운 내용을 독립된 기억으로 보존하지 못한다.
2:30–3:40	핵심 아이디어	Raw Trace → Wiki → Skill의 세 층을 분리한다.
3:40–5:20	수식	Skill은 검증 후 채택·rollback하지만 Wiki는 계속 누적된다.
5:20–8:00	시스템	실행·분석·제안·검증이 반복되는 continual loop다.
8:00–11:30	실험과 결과	5개 benchmark, 5개 모델에서 일관된 개선과 scaling 경향을 보인다.
11:30–14:25	전이·ablation·사례	경험은 모델 간 전이되고, 실패도 다음 제안의 재료가 된다.
14:25–15:00	한계·결론	Skill Evolution을 knowledge accumulation 문제로 재정의한다.

> **발표 리듬**
>
> 첫 3분은 문제와 비유로 청중을 끌어오고, 3–8분은 구조와 수식으로 신뢰를 만들며, 8분 이후에는 숫자와 사례로 '정말 효과가 있는가'를 증명합니다.

0:00–1:10 | 오프닝 - AI 에이전트에게도 경력이 쌓일까요?

사람의 업무 경험과 에이전트의 실행 trace를 대비시키며 문제의식을 만듭니다.

[발표자 큐] 제목을 가리키고, 청중을 보며 질문으로 시작합니다.

안녕하세요. 오늘 소개드릴 논문은 Google Research와 Virginia Tech 연구진의 WikiSkill: Compiling Agent Experience into Persistent Knowledge for Skill Evolution입니다.

발표를 시작하면서 한 가지 질문을 드리고 싶습니다. AI 에이전트도 일을 오래 하면 경력이 쌓일까요?

사람에게 일을 처음 시켜 보면 실수를 많이 합니다. 하지만 같은 업무를 1년, 3년 반복한 사람은 처음과 다릅니다. 지난번에 무엇이 실패했는지, 어떤 순서가 더 잘 맞는지, 겉보기에는 다른 문제의 공통 원인이 무엇인지 기억하기 때문입니다.

그런데 현재의 AI 에이전트는 조금 이상합니다. 수십 번, 수백 번 작업을 수행하고 성공과 실패를 모두 경험해도, 다음 iteration에는 그 경험 전체가 지속적인 지식으로 남지 않는 경우가 많습니다. 현재의 Skill 파일만 가지고 다시 출발하는 것입니다.

기존 Skill Evolution도 실행 trace를 보고 Skill을 개선합니다. 그러나 논문의 문제 제기는 분명합니다. Skill을 발전시키는 과정에서 얻은 통찰이 optimization history 곳곳에 흩어져 있고, 별도의 진화하는 knowledge representation으로 유지되지 않는다는 것입니다.

> **오프닝의 전환 문장**
>
> "Agent의 경험을 일회성 로그가 아니라, 계속 축적되는 지식으로 컴파일할 수는 없을까?" 이 질문에서 WikiSkill이 시작됩니다.

1:10–2:30 | 연구 배경 - 기존 Skill Evolution의 빈자리

Skill은 모델 parameter가 아니라, 특정 업무를 위한 재사용 가능한 절차적 지식입니다.

[발표자 큐] Skill을 '사실 지식'이 아니라 '행동 규칙'으로 정의합니다.

이 논문에서 Skill은 모델 parameter를 다시 학습하는 것이 아닙니다. 특정 업무를 수행하기 위한 instruction, workflow, script, applicability condition 같은 재사용 가능한 절차적 지식입니다.

기존 방법도 agent를 task에 투입하고, trace를 얻고, 성공과 실패를 분석하고, Skill을 업데이트합니다. 대표적으로 EvoSkill, Trace2Skill, SkillOpt 같은 접근이 있습니다.

하지만 저자들은 한 단계 더 묻습니다. '이전 iteration에서 무엇을 배웠는지는 어디에 있는가?' 직원이 매주 업무 매뉴얼을 수정하면서도, 왜 그 문장을 추가했는지, 지난번에 무엇이 실패했는지 기록한 업무일지는 계속 버리는 것과 같습니다.

발표자의 문제 정의

> **"Skill은 업데이트되지만, 경험에서 얻은 통찰이 독립적이고 계속 진화하는 지식으로 남지 않는다."**

2:30–3:40 | 핵심 아이디어 - Raw Trace → Wiki → Skill

세 층의 역할을 분리하면, 경험과 현재 행동 규칙을 동시에 관리할 수 있습니다.

[발표자 큐] 세 계층을 손가락으로 차례로 가리키며 설명합니다.

WikiSkill의 핵심은 세 단어로 기억할 수 있습니다. Raw, Wiki, Skill.

첫 번째 Raw Layer는 agent가 실제로 어떻게 문제를 풀었는지에 대한 원본 execution trace입니다. 어떤 observation을 보았고, 어떤 tool을 호출했고, 어떤 결과가 나왔고, 성공했는지 실패했는지가 모두 들어 있습니다. 이 층은 Write Once, Permanent입니다.

두 번째 Wiki Layer는 raw trace를 그대로 쌓는 곳이 아닙니다. 반복되는 실패 원인, 성공 전략, 해결 workaround, 과거 proposal의 성공과 실패를 구조화된 pattern과 evolution log로 정리합니다. 핵심은 Compounding, Never Reset입니다.

세 번째 Skill Layer는 실제 task 수행에 사용하는 현재의 행동 규칙입니다. Skill은 수정할 수 있고, 성능이 나빠지면 이전 상태로 되돌릴 수 있습니다.

> **세 층의 한 줄 정의**
>
> Raw는 '무슨 일이 있었는가', Wiki는 '그 경험에서 무엇을 배웠는가', Skill은 '그래서 다음에는 어떻게 행동할 것인가'입니다.

3:40–5:20 | 핵심 수식 - Wiki는 기억하고 Skill은 경쟁합니다

수식은 복잡해 보이지만, '실행 → 기록 → 제안 → 검증'의 순서를 나타냅니다.

데이터 분할

$$D = D_{train} \cup D_{val} \cup D_{test}$$

학습·검증·테스트를 분리하여, 새로운 Skill이 실제로 좋아졌는지 검증합니다.

실행 trace 생성

$$\tau_i \sim \pi(x_i; S_{(k\ 1)})$$

현재 승인된 Skill $S_{(k\ 1)}$로 task를 수행하고, observation과 action의 전체 과정을 기록합니다.

Wiki 누적

$$W'_k \leftarrow M_{WM}(W_{(k\ 1)}, T_{sample,k})$$

Wiki Maintainer가 기존 Wiki와 샘플 trace를 바탕으로 root cause와 반복 패턴을 추가합니다.

Skill 제안과 적용

$$P_k \leftarrow M_P(W'_k, S_{(k\ 1)}, T_{train,k}) \;/\; S'_k = Apply(S_{(k\ 1)}, P_k)$$

Skill Proposer가 Wiki와 trace에 근거해 후보 Skill을 만들고 적용합니다.

검증과 gating

$$S_k = S'_k \text{ if } R(T_{val},k) > R_{best}; \text{ otherwise } S_{(k-1)}$$

성능이 좋아졌을 때만 채택하고, 그렇지 않으면 Skill만 rollback합니다.

가장 중요한 비대칭성

Skill은 rollback하지만 Wiki는 rollback하지 않습니다. 틀린 답은 버리되, 왜 틀렸는지에 대한 경험은 버리지 않는다는 뜻입니다.

5:20–8:00 | 시스템 구조 - 네 명의 직원이 협업한다고 생각하기

복잡한 다이어그램을 실행자·분석자·제안자·품질관리자의 협업으로 번역합니다.

Step 1. Inference Agent - 현장 직원

Inference Agent는 실제 training task를 수행합니다. 현재 승인된 Skill을 system prompt에 직접 주입받아 사용하고, 그 과정 전체를 Raw Layer에 기록합니다. 여기서 의도적인 설계가 하나 있습니다. Inference Agent는 training rollout 동안 Wiki를 직접 읽지 않습니다.

Step 2. Wiki Maintainer - 사후 분석 담당자

Wiki Maintainer는 성공 trace와 실패 trace 일부를 샘플링해 분석합니다. 실패에서는 '왜 틀렸는가'를, 성공에서는 '무엇이 효과적이었는가'를 찾습니다. 단순히 요약하는 것이 아니라 root-cause analysis를 수행하고, patterns와 logs에 구조화합니다.

Step 3. Skill Proposer - 업무 프로세스 개선 담당자

Skill Proposer는 현재 Skill뿐 아니라 Wiki와 필요한 raw trace를 함께 봅니다. 이번 iteration에서 가장 중요한 failure mode가 무엇인지 판단하고, 한 번에 하나의 atomic proposal을 만듭니다. 거대한 Skill 전체를 무작정 다시 쓰는 방식이 아닙니다.

Step 4. Gating & Rollback - 품질관리자

새 Skill은 validation set에서 시험합니다. 좋아졌으면 Accept, 나빠졌으면 Reject하고 이전 Skill로 돌아갑니다. 다만 rejected proposal과 그 결과는 Wiki의 history에 남습니다.

반복되는 continual loop

실행 → 기록 → 분석 → 지식화 → Skill 제안 → 검증 → 다시 실행. WikiSkill은 단순한 prompt optimization이 아니라, agent experience를 persistent knowledge로 컴파일하는 loop입니다.

8:00–9:00 | 실험 설정 - 특정 task 하나에서만 되는가?

다양한 업무·모델 조합에서 framework의 일반성을 확인합니다.

저자들은 총 5개 benchmark와 5개 model을 사용합니다. 수학 추론, 웹 검색, spreadsheet manipulation, long-context document QA, interactive embodied task를 포함합니다.

범주	Benchmark 예시	보는 능력
수학 추론	LiveMathematicianBench, LiveMath	계산·추론 절차
웹 검색	SealQA	검색·근거 수집
스프레드시트	SpreadsheetBench	도구 조작·검증
장문 문서	OfficeQA	문서 탐색·질의응답
상호작용 환경	ALFWorld	상태 추적·행동 계획

비교 대상은 No Skill, Trace2Skill, EvoSkill, SkillOpt, WikiSkill입니다. 각 방법의 전체 evolution process를 세 번 독립적으로 반복하고 평균 test performance를 보고하며, paired bootstrap test도 수행합니다.

9:00–10:25 | 핵심 결과 1 - 다양한 모델에서 일관된 개선

숫자 하나보다 '모델과 task를 가로지르는 안정성'을 강조합니다.

[발표자 큐] 그래프에서는 WikiSkill을 가리키고, 모든 숫자를 읽기보다 증가 폭과 모델별 추세를 말합니다.

결과에서 먼저 보실 것은 WikiSkill의 평균 성능입니다. Qwen 3.5 4B는 26.2에서 38.5, 9B는 29.9에서 47.4, 27B급 모델은 39.4에서 63.3으로 올라갑니다. Gemma-4-31B는 41.3에서 54.9, Gemini-3.5-Flash는 49.5에서 68.1입니다.

모델	No Skill 평균	WikiSkill 평균	개선 폭
Qwen-3.5-4B	26.2	38.5	+12.3
Qwen-3.5-9B	29.9	47.4	+17.5
Qwen-3.6-27B	39.4	63.3	+23.9
Gemma-4-31B	41.3	54.9	+13.6
Gemini-3.5-Flash	49.5	68.1	+18.6

대표 task에서는 Gemini-3.5-Flash의 LiveMath가 33.0에서 72.6, Spreadsheet가 50.5에서 76.6으로 올라갑니다. Qwen-3.6-27B의 ALFWorld는 52.8에서 77.6입니다.

> **결과 해석**
>
> WikiSkill의 장점은 최고점 하나가 아니라, 서로 다른 model–task 조합에서 상대적으로 일관된 개선을 보였다는 점입니다. 기존 방법은 어떤 조합에서 좋아졌다가 다른 조합에서는 오히려 떨어지는 경우도 있습니다.

10:25–11:30 | 핵심 결과 2 - 모델이 클수록 경험 활용의 이득도 커진다

Skill Evolution과 model scaling은 경쟁 관계가 아니라 상호 보완적일 수 있습니다.

Qwen 계열에서 No Skill 대비 WikiSkill의 개선 폭은 4B에서 +12.3 point, 9B에서 +17.5 point, 27B에서 +23.9 point입니다. 모델 capability가 높아질수록 구조화된 경험을 활용하는 이득도 커지는 경향입니다.

흥미로운 비교도 있습니다. Qwen-3.5-9B + WikiSkill은 평균 47.4%인데, 훨씬 큰 Qwen-3.6-27B가 Skill 없이 수행하면 39.4%입니다. 즉, 작은 모델과 좋은 절차 지식의 조합이 큰 모델과 절차 지식 없음보다 더 잘할

수 있습니다.

한 줄 해석

> **“Model capability와 evolved procedural knowledge는 서로를 대체하기보다 보완한다.”**

11:30–12:30 | 전이 - 한 에이전트의 경력을 다른 에이전트가 물려받을 수 있는가?

외부화된 procedural knowledge가 모델 간 전이 가능성을 만듭니다.

한 모델이 배운 Skill을 다른 모델에게 전달할 수 있을까요? 일부 조건에서 transferred Skill은 해당 모델이 자기 경험만으로 만든 self-evolved Skill보다도 높은 성능을 보입니다.

조건 예시	No Skill	Self-evolved	Transferred	읽는 포인트
전이 사례 A	24.3	33.6	50.5	외부화된 노하우가 자기 경험을 앞섬
전이 사례 B	33.9	56.7	73.7	전이 후 추가 상승

이 결과의 의미는 Skill이 특정 model parameter 안에만 존재하는 tacit knowledge가 아니라, 텍스트 형태의 procedural knowledge로 외부화되기 때문에 다른 모델에서도 재사용될 가능성이 생긴다는 것입니다. 고참 직원의 업무 노하우를 신입 직원에게 인수인계할 수 있게 된 셈입니다.

12:30–13:15 | Ablation - Wiki는 누구에게나 직접 보여주면 되는가?

기억하는 층과 행동하는 층을 분리한 설계의 이유를 보여줍니다.

Wiki가 좋다면 모든 Agent가 Wiki를 읽으면 더 좋을 것 같지만, ablation 결과는 단순하지 않습니다. No Skill은 40.4, Inference Agent만 Wiki를 보게 하면 45.3, 다른 변형은 48.7, 두 agent가 Wiki를 사용하면 60.9, 기본 WikiSkill 구조는 63.7입니다.

구성	성능	해석
No Skill	40.4	기준선
Inference Agent만 Wiki 사용	45.3	행동 단계에 원문 지식이 직접 들어감
Wiki 활용이 제한된 변형	48.7	부분적인 이득
두 agent가 Wiki 사용	60.9	분석과 제안에 기억을 활용
기본 WikiSkill	63.7	Skill Proposer 중심의 분리된 활용

> **예상 오해 차단**
>
> Wiki는 모든 정보를 inference prompt에 그대로 붙이는 RAG 메모리가 아닙니다. Wiki는 분석과 Skill 제안에 사용되고, 실제 행동은 검증된 Skill을 중심으로 수행하도록 역할을 분리합니다.

13:15–13:50 | 사례 연구 - 실패도 자산이 된다

'Reject'가 학습의 끝이 아니라 다음 proposal의 시작이 되는 장면입니다.

ALFWorld의 사례에서 첫 iteration의 새로운 Skill은 validation 성능을 떨어뜨립니다. 그래서 REJECTED 됩니다. 기존 방식이라면 여기서 이야기가 끝날 수 있습니다.

하지만 WikiSkill에서는 실패 원인이 pattern과 log에 남습니다. 다음 iteration에서 Skill Proposer는 그 기록을 다시 읽고 break-repetition-loop와 같은 더 구체적인 rule을 제안합니다. 예를 들면 'Each Operation Type ONCE Per Item', 즉 하나의 item에 동일한 operation을 불필요하게 반복하지 말라는 규칙입니다.

> **사례의 핵심**
>
> 첫 번째 실패가 Wiki에 보존되어 있었기 때문에 두 번째 proposal이 더 구체적으로 진화할 수 있었습니다. WikiSkill에서 실패는 삭제해야 할 오류가 아니라 다음 Skill을 만드는 training material입니다.

13:50–14:25 | 장점과 한계 - 무엇을 주장하고, 무엇을 아직 모르는가

강점만 말하지 않고 비용과 열린 문제를 함께 밝혀 신뢰를 높입니다.

구분	발표에서 말할 내용
장점	실행 경험을 auditable한 외부 지식으로 남길 수 있고, Skill을 검증·rollback할 수 있으며, 다른 모델로 transfer할 가능성이 생깁니다.
비용	Wiki 유지, trace 분석, proposal 생성, validation 반복 때문에 단순 inference보다 token과 rollout 비용이 추가됩니다.
의존성	Wiki Maintainer와 Skill Proposer의 품질은 underlying model의 분석·추론 능력에 영향을 받을 수 있습니다.
열린 문제	Wiki가 장기 누적되면 중복 패턴, 오래된 규칙, 충돌하는 규칙을 어떻게 관리할지가 중요해집니다.

예상 질문을 미리 막는 문장

> **"이 논문이 말하는 것은 무한한 자기개선이 아닙니다."**
>
> WikiSkill은 모델 parameter를 자동으로 재학습한다는 주장이 아니라, agent의 실행 경험을 구조화된 외부 지식으로 축적하고, 그 지식을 이용해 Skill을 제안·검증하는 프레임워크입니다. 따라서 성능 향상에는 추가 비용과 proposer 품질이 함께 관여합니다.

14:25–15:00 | 최종 결론 - Skill Evolution을 '기억의 구조'로 다시 보기

마지막에는 세 단어와 한 문장만 남깁니다.

[발표자 큐] 마지막 페이지에서 세 단어를 천천히 읽고, 마지막 한 문장 뒤에 잠시 멈춥니다.

에이전트가 task를 수행하면 경험이 생깁니다. WikiSkill은 이 경험을 버리지 않고 구조화하여 지식으로 만듭니다. 그리고 그 지식을 검증 가능한 procedural rule인 Skill로 변환합니다.

Experience → Knowledge → Skill

이 논문의 가장 중요한 기여는 새로운 Skill generator 하나를 제안했다는 것보다, agent가 어떻게 경험을 축적해야 하는가라는 문제를 Skill optimization에서 knowledge accumulation 문제로 바꾸었다는 데 있습니다.

잘못된 Skill은 버릴 수 있습니다. 그러나 그 Skill을 만들게 했던 경험까지 버려서는 안 됩니다. 이런 지속적이고 구조적인 경험 축적이 장기적으로는 스스로 개선되고, 다른 모델에게도 노하우를 전달할 수 있는 Agent의 기반이 될 수 있습니다.

> **최종 한 문장**
>
> "WikiSkill이 에이전트에게 주는 것은 더 좋은 Skill 하나가 아니라, 경험을 잊지 않고 다음 Skill로 바꾸는 '경력의 구조'입니다."

이상으로 발표를 마치겠습니다. 감사합니다.

부록 | 예상 질문 방지 문장과 Q&A; 치트시트

질문이 나오기 전에 발표 중 한두 문장으로 미리 정리해 두면 좋은 포인트입니다.

Q. WikiSkill은 모델을 추가 학습하나요?

A. 핵심은 parameter update가 아니라 외부화된 procedural knowledge의 축적과 Skill 파일의 진화입니다.

Q. Wiki와 Raw Trace의 차이는 무엇인가요?

A. Raw Trace는 원본 실행 기록이고, Wiki는 여러 trace에서 반복 패턴과 원인을 구조화한 해석 층입니다.

Q. 실패한 proposal은 완전히 삭제되나요?

A. Skill은 rollback하지만 proposal 결과와 실패 원인은 Wiki history에 남습니다.

Q. Inference Agent가 Wiki를 직접 읽지 않는 이유는요?

A. Wiki는 풍부한 분석 지식이기 때문에 행동 단계에 그대로 넣으면 복잡성을 늘릴 수 있어, 분석·제안 층에서 활용하도록 분리합니다.

Q. 단순한 RAG와 무엇이 다른가요?

A. 목표가 문서 검색이 아니라 실행 경험에서 root cause, pattern, evolution log를 만들고 검증된 Skill로 컴파일하는 continual loop라는 점이 다릅니다.

Q. 결과를 모든 task에 일반화할 수 있나요?

A. 논문은 5개 benchmark와 5개 model에서 효과를 보였지만, 모든 업무에 대한 보장은 아니며 비용·지식 관리가 열린 문제입니다.

Q. 가장 중요한 설계 포인트는 무엇인가요?

A. 경험의 원본, 구조화된 기억, 현재 행동 규칙을 분리하고, Skill만 gating하는 비대칭성입니다.

발표자가 반드시 힘을 줄 5문장

1. 현재의 Agent는 task를 반복하면서도 그 경험을 충분히 축적하지 못합니다.

2. WikiSkill은 지식을 Raw Trace, Wiki, Skill의 세 층으로 분리합니다.

3. Skill은 rollback할 수 있지만 Wiki는 reset하지 않습니다.

4. 따라서 실패한 Skill도 실패의 원인이라는 형태로 다음 Skill의 재료가 됩니다.

5. WikiSkill은 Skill Evolution을 단순한 prompt 수정 문제가 아니라 지속적인 knowledge accumulation 문제로 재정의합니다.

자료 기준

제공된 원문: WikiSkill: Compiling Agent Experience into Persistent Knowledge for Skill Evolution. 발표 대본과 수치는 참조 대화에서 복원한 초안 및 제공 논문·포스터를 기준으로 정리했습니다.